Inyección de comandos del sistema operativo (OS Command Injection)

1. Descripción

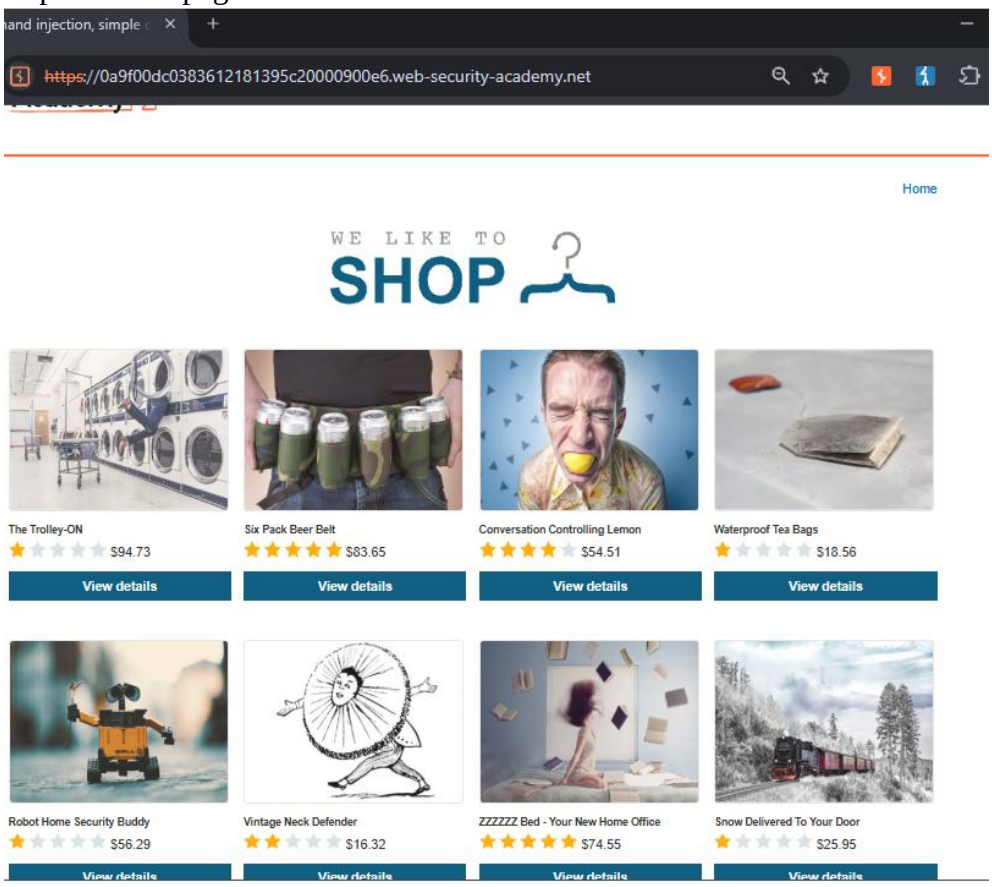
La aplicación presentaba una vulnerabilidad de inyección de comandos del sistema operativo (OS Command Injection) en la funcionalidad de consulta de stock.

La solicitud `POST /product/stock` recibía los parámetros `productId` y `storeId`. Al modificar el valor de `storeId` y concatenar el separador de comandos `|` seguido de `whoami`, fue posible provocar la ejecución de un comando del sistema operativo en el servidor.

La respuesta incluyó el nombre de la cuenta del sistema que ejecutó el comando, lo que confirmó que la entrada proporcionada por el usuario podía alterar la operación ejecutada por el servidor.

2. Evidencia

Se recopilaron las siguientes evidencias durante la prueba:

- Captura de la página inicial de la tienda.
- 

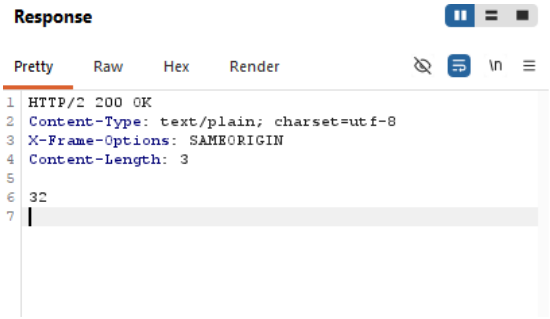
- Solicitud original `POST /product/stock`, con los parámetros `productId` y `storeId`.

Request

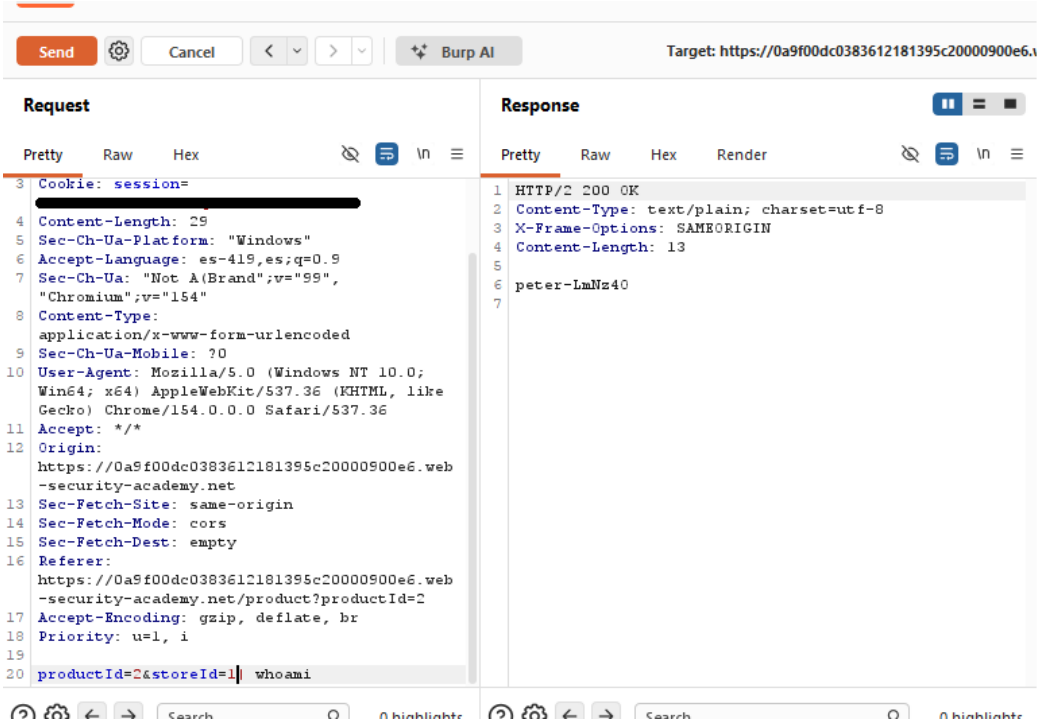
Pretty Raw Hex

```
1  POST /product/stock HTTP/2
2  Host:
   0a9f00dc0383612181395c20000900e6.web-security-academy.net
3  Cookie: session=[redacted]
4  Content-Length: 21
5  Sec-Ch-Ua-Platform: "Windows"
6  Accept-Language: es-419,es;q=0.9
7  Sec-Ch-Ua: "Not A(Brand";v="99", "Chromium";v="154"
8  Content-Type: application/x-www-form-urlencoded
9  Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0
   Safari/537.36
11 Accept: */*
12 Origin:
   https://0a9f00dc0383612181395c20000900e6.web-security-aca
   demy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
   https://0a9f00dc0383612181395c20000900e6.web-security-aca
   demy.net/product?productId=2
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 productId=2&storeId=1
```

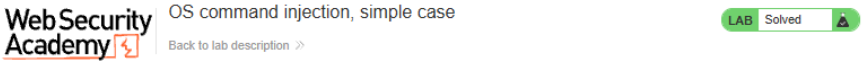
- Respuesta original del servidor con estado `200 OK` y el valor de stock `32`.



- Solicitud modificada con el payload `storeId=1| whoami` y respuesta que muestra la salida del comando.



- Confirmación de que el laboratorio de PortSwigger fue resuelto.





3. Pasos para reproducir

1. Acceder al laboratorio de OS Command Injection de PortSwigger Web Security Academy.
2. Seleccionar un producto y utilizar la funcionalidad de consulta de stock.
3. Interceptar o localizar la solicitud `POST /product/stock` mediante Burp Suite.
4. Enviar la solicitud original a Repeater y comprobar que el servidor devuelve una respuesta normal.
5. Modificar el parámetro `storeId`, agregando el separador `|` y el comando `whoami`. Por ejemplo:

   ```
   productId=2&storeId=1| whoami
   ```

6. Enviar la solicitud modificada y observar que la respuesta incluye el nombre de la cuenta del sistema que ejecutó el comando.

4. Impacto

Confidencialidad: potencialmente afectada. La ejecución de comandos podría permitir leer archivos o acceder a información disponible para la cuenta del proceso del servidor.

Integridad: potencialmente afectada. Según los permisos disponibles, un atacante podría modificar o eliminar archivos y alterar recursos accesibles para el proceso.

Disponibilidad: potencialmente afectada. La ejecución de comandos podría interrumpir el servicio o consumir recursos del sistema.

El resultado confirmado fue la ejecución del comando `whoami` y la devolución de la identidad de la cuenta del sistema. No se verificó de forma independiente el acceso a información sensible, la modificación de archivos ni la interrupción del servicio.

5. Causa raíz

La causa raíz es el tratamiento inseguro de datos controlados por el usuario en una operación que ejecuta comandos del sistema operativo.

La aplicación permitía que metacaracteres del intérprete de comandos modificaran la instrucción ejecutada, en lugar de tratar la entrada exclusivamente como un dato.

6. Remediación

- Evitar la ejecución de comandos del sistema operativo cuando la operación pueda realizarse mediante una API o biblioteca.
- Si es imprescindible ejecutar un proceso externo, utilizar mecanismos que separen el ejecutable de sus argumentos y evitar la interpretación de entradas mediante un shell.
- Validar `storeId` en el servidor mediante una lista permitida de identificadores válidos.
- Ejecutar la aplicación con el mínimo privilegio necesario.
- Restringir el acceso del proceso a archivos, recursos y servicios de red.
- Incorporar pruebas de seguridad para comprobar que las entradas del usuario no puedan modificar la ejecución de comandos.

7. Severidad y CVSS

Severidad provisional: alta, debido a que se confirmó la ejecución de un comando del sistema operativo desde una entrada controlada por el usuario.

No se asigna un puntaje CVSS definitivo únicamente a partir de esta prueba. La puntuación depende de las condiciones de explotación, los privilegios requeridos, el alcance y el impacto real sobre la confidencialidad, integridad y disponibilidad.

8. Alcance de la prueba

La vulnerabilidad fue reproducida en un laboratorio educativo de PortSwigger Web Security Academy. Las pruebas se limitaron al entorno del laboratorio y no implican que un sistema de producción específico presente la misma condición.